

Einführung mit Turtle

1. Was ist «Programmieren»?

Ein Computer ist zunächst einfach mal eine Maschine, die wahnsinnig schnell arbeiten (rechnen) kann. Damit Sie ein Spiel spielen können, muss irgend jemand dem Computer also sagen, was er denn genau zu rechnen habe. Ihr Hund versteht Sie nicht, wenn Sie mit ihm sprechen wie mit einem Kollegen - genauso versteht der Computer Sie nicht, wenn Sie ihm Ihre Wünsche einfach in normalem Text schreiben. Es braucht in beiden Fällen eine **formalisierte Sprache**.

Wir geben dann Kommandos - sowohl dem Hund (Sitz!) als auch dem Computer (forward()).



Dem Computer beibringen, was die einzelnen Kommandos bedeuten sollen und danach die richtigen Kommandos in der richtigen Reihenfolge hinschreiben - das nennt man **programmieren**.

Diese Kommandos sind also in einer Sprache geschrieben, so wie wir auch z.B. Deutsch, Französisch, Mandarin oder Spanisch sprechen. Und wie es auf der Welt viele verschiedene Sprachen gibt, gibt es auch in der Welt der Computer viele Programmiersprachen (siehe Abbildung 1).

Wir schauen uns in diesem Kurs die Programmiersprache **Python** vertieft an, da sie einfach für Einsteiger ist und viele Anwendungsgebiete hat. Doch worin unterscheiden sich die verschiedenen Sprachen und warum gibt es so viele?

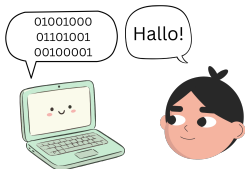


Abbildung 1: Logos von Programmiersprachen



Programmiersprachen unterscheiden sich ähnlich wie unsere gesprochenen Sprachen voneinander: Es gibt verschiedene **Regeln** (Grammatik, Rechtschreibung, Aufbau eines Satzes, ...). Es gibt so viele Sprachen, weil sich je nach Situation, was man programmieren möchte, eine besser eignet als die andere.

Zum Beispiel wenn man Programme für Android-Geräte entwickelt, braucht man oft die Programmiersprache «Kotlin». Wenn man hingegen mehrheitlich statistische Berechnungen machen möchte, braucht man eher die Sprache «R».



Programmieren bedeutet also, einer Maschine Befehle zu erteilen und sie damit zu steuern. Wie wir aber auch im letzten Thema gelernt haben, speichert der Computer alle Informationen in Form von 0 und 1. Alles (egal ob Texte, Bilder oder Musik) wird im Inneren des Computers als eine lange Folge von 0 und 1 dargestellt. Doch für uns Menschen wäre es mühsam, mit einem Computer direkt in dieser Sprache aus lauter 0 und 1 zu sprechen.

Darum wurden Programmiersprachen entwickelt. Sie dienen als **Übersetzungsschicht** zwischen Mensch und Maschine. Wir können damit Anweisungen in einer Sprache formulieren, die für uns verständlich ist - und der Computer kann sie dann Schritt für Schritt in seine „Sprache“ aus 0 und 1 umsetzen.



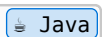
Damit das funktioniert, brauchen wir eine **Entwicklungsumgebung**. Das ist ein Programm, das uns beim Schreiben, Testen und Ausführen von Code hilft. So können wir dem Computer Befehle geben, ohne uns um die komplizierten Binärcodes kümmern zu müssen.

Hier sehen Sie zum Beispiel vier Mal das gleiche Programm, aber in verschiedenen Programmiersprachen geschrieben:

```
1 x = int(input("Zahl: "))
2 y = x * x
3 print("Quadrat:", y)
```



```
1 Scanner s = new Scanner(System.in);
2 int x = s.nextInt();
3 int y = x * x;
4 System.out.println("Quadrat: " + y);
```



```
1 let mut s = String::new();
2 std::io::stdin().read_line(&mut s).unwrap();
3 let x: i32 = s.trim().parse().unwrap();
4 println!("Quadrat: {}", x * x);
```

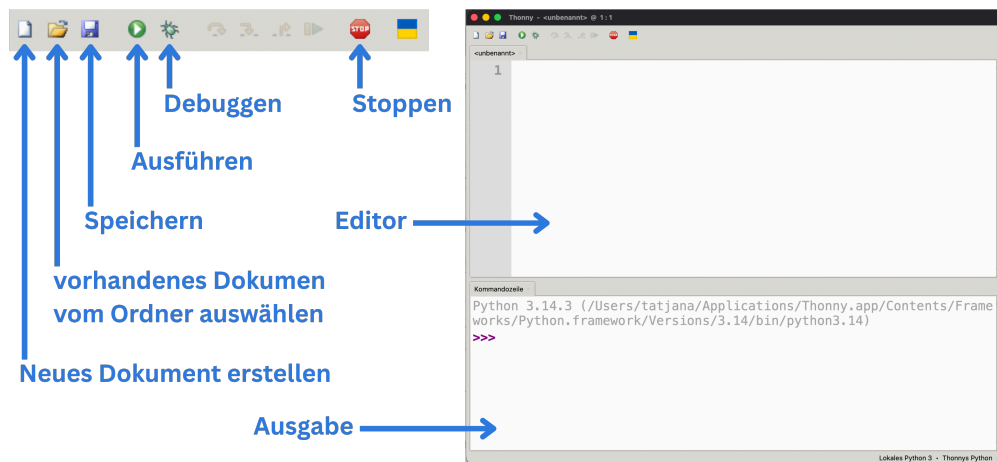


```
1 x <- as.integer(readline("Zahl: "))
2 y <- x * x
3 print(paste("Quadrat:", y))
```



2. Unser erstes Setup: Thonny und Turtle

Wie es viele verschiedene Programmiersprachen gibt, gibt es auch eine Menge unterschiedlicher Entwicklungsumgebungen. Wir werden im Unterricht die Umgebung **Thonny** brauchen, da es überschaubar, leicht zu installieren und nicht zu anspruchsvoll ist. Hier eine kurze Übersicht und Erklärung der Symbole:



Von links nach rechts, oben nach unten die einzelnen Symbole erklärt:

- Sie können wie im Word ein **neues Dokument erstellen**, um mit dem Programmieren starten.
- Sie können auch ältere Dokumente **wieder öffnen**.
- **Speichern** Sie Ihr Dokument ab, damit Sie Ihren Code später wieder ansehen können.
- **Ausführen** bedeutet, dass Sie Ihren Code dem Computer als Befehl geben. Er wird dann versuchen, Ihre Befehle (die Code-Zeilen) auszuführen.
- **Debuggen** ist ein Tool, um den Ablauf Ihres Codes Schritt für Schritt nachzuvollziehen.
- Sie können den Computer zwingen, die Ausführung zu **stoppen**, wenn er z.B. etwas unendlich oft wiederholt. So können Sie verhindern, dass evtl. Ihr Computer abstürzt.
- Im **Editor** schreiben Sie Ihren Code.
- In der **Ausgabe** sehen Sie das Resultat, nachdem der Computer Ihre Befehle ausgeführt hat.

Das turtle Modul

Unsere erste Programmiererfahrung wird darin bestehen, einer kleinen Schildkröte Befehle zu erteilen, damit sie auf dem Bildschirm herumläuft.



Für das brauchen wir das `turtle` Modul. In diesem Modul befinden sich die Befehle für die Schildkröte. Sie können sich vorstellen, dass diese Befehle nicht «normaler» Python Code sind. Die meisten Informatiker möchten ja nicht den ganzen Tag irgendeine Schildkröte herumlaufen lassen. Darum sind diese Befehle in einem Paket (Modul) verpackt und wenn man sie brauchen möchte, muss man dieses Paket halt zuerst öffnen (importieren).

In Thonny, falls Sie die Schildkröte herumkommandieren möchten, müssen Sie also am Anfang Ihres Codes jedes Mal die folgende Zeile hinschreiben:

```
1 from turtle import *
```



Wobei:

- `from` ist ein spezielles Wort in Python (Keyword), das man fürs Importieren von Modulen braucht.
- mit `turtle` geben wir den Namen des Moduls an, das wir importieren möchten.
- `import` ist wieder ein spezielles Wort in Python, das wir für das Importieren brauchen.
- der Stern `*` signalisiert, dass wir von diesem Modul **jeden** Befehl auspacken möchten.

Man kann also die Codezeile wie folgt lesen: «Vom Paket `turtle` importiere bitte alle Befehle, die da drin sind».

Es gibt einige Befehle, um die Schildkröte auf Ihrem Bildschirm herumlaufen zu lassen. Der Computer funktioniert aber nicht wie wir Menschen und würde es verstehen, wenn Sie einen kleinen Fehler in der Rechtschreibung machen.



Daher ist es beim Programmieren sehr wichtig, den Code **fehlerfrei** zu schreiben! Rechtschreibfehler kann Ihr Computer nicht erkennen.

Hier finden Sie eine Liste der gängigsten Turtle-Befehle.

Befehl	Beschreibung
<code>forward(n)</code>	Bewegt die Turtle um n Pixel vorwärts.
<code>back(n)</code>	Bewegt die Turtle um n Pixel zurück.
<code>left(w)</code>	Dreht die Turtle um w Grad nach links.
<code>right(w)</code>	Dreht die Turtle um w Grad nach rechts.
<code>goto(x,y)</code>	Die Turtle läuft direkt zum Punkt (x, y).
<code>shape("form")</code>	Ändert das Aussehen der Turtle zu der angegebenen Form. Mögliche Formen sind: • turtle • arrow • circle • square • triangle • classic
<code>penup()</code>	Die Turtle hebt den Stift hoch und wird nach diesem Befehl nicht mehr zeichnen.
<code>pendown()</code>	Die Turtle legt den Stift wieder runter und wird ab jetzt beim Bewegen wieder mitzeichnen.
<code>dot(d)</code>	Zeichnet einen ausgefüllten Kreis mit Durchmesser d Pixel dort, wo sich die Turtle befindet.
<code>circle(r)</code>	Die Turtle bewegt sich in einem Kreis mit Radius r nach links.
<code>circle(r,w)</code>	Die Turtle bewegt sich in einem Kreisbogen mit dem Radius r und Winkel w nach links.
<code>pencolor("farbe")</code>	Ändert die Farbe des Stiftes, mit der die Turtle zeichnet. Es gibt viele mögliche Farben, z. B.: • coral • lime • plum • weitere unter trinket.io/docs/colors
<code>pensize(d)</code>	Setzt die Dicke der Linie, die von der Turtle gezeichnet wird.
<code>hideturtle()</code>	Versteckt die Turtle.
<code>showturtle()</code>	Zeigt die Turtle wieder.
<code>fillcolor("farbe")</code>	Legt die Farbe fest, falls die Turtle etwas mit Farbe ausfüllen möchte.
<code>begin_fill()</code>	Beginnt damit, eine gefüllte Form zu zeichnen.
<code>end_fill()</code>	Beendet die Figur und füllt sie mit der vorher definierten Farbe aus.
<code>done()</code>	Sollte am Ende des Codes stehen, damit das Fenster nicht einfriert.

Hier sehen Sie Beispiele, wie man diese Befehle in Thonny eingeben kann:

```
1 from turtle import *
2 forward(10)
3 turn(90)
4 forward(20)
5 done()
```



```
1 from turtle import *
2 shape("square")
3 forward(100)
4 circle(50)
5 done()
```



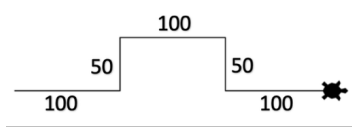
In Python sind leere Zeilen kein Problem. Das ändert den Code nicht. Wenn also beim obigen Beispiel zwischen Zeile 1 und 2 eine leere Zeile wäre, würde der Code trotzdem funktionieren. So können Sie Ihren Code übersichtlicher gestalten, falls Sie das wünschen.

In der Informatik wird manchmal für das, was wir bis jetzt Befehl genannt haben auch der Ausdruck **Funktion** oder **Methode** verwendet. Lassen Sie sich davon nicht verwirren:)

Achten Sie bei den folgenden Aufgaben immer auf die Form Ihrer Turtle, damit Ihre Lösung genau die gleiche ist, wie in der Aufgabe angegeben!

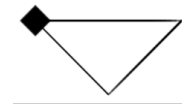
Aufgabe 1

Versuchen Sie, die Turtle anzuleiten, folgende Figur zu zeichnen. Achten Sie auf die Form der Turtle.



Aufgabe 2

Zeichnen Sie ein gleichschenkelig-rechtwinkliges Dreieck mit der Turtle. Die zwei kurzen Seiten sind je 100 lang.



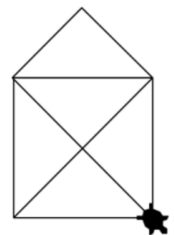
Aufgabe 3

Verwenden Sie den Befehl `dot()`, um folgende Figur zu zeichnen.



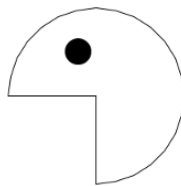
Aufgabe 4

Ziel beim «Haus des Nikolaus» ist es, das besagte Haus in einem Linienzug aus genau 8 Strecken zu zeichnen, ohne dabei eine Strecke zweimal zu durchlaufen. Versuchen Sie es mit der Turtle.



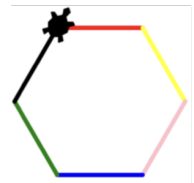
Aufgabe 5

Verwenden Sie `circle(r,w)` und `dot(d)` so, damit Sie damit die folgende Figur zeichnen.



Aufgabe 6

Zeichnen Sie mit der Turtle ein regelmässiges Sechseck mit der Stiftbreite 7 und wählen Sie für jede Seite eine andere Farbe.



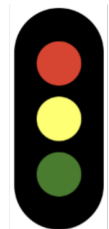
Aufgabe 7

Zeichnen Sie die folgende Figur. Beachten Sie, dass am Ende die Turtle verschwinden soll. Die verschiedenen Farben dürfen sie selbst wählen.



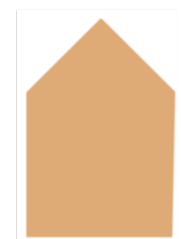
Aufgabe 8

Zeichnen Sie eine Ampel. Das schwarze, abgerundete Rechteck können Sie mit der Stiftbreite 80 zeichnen, die Kreisfläche mit `dot(40)`. Am Ende soll die Turtle verschwinden.



Aufgabe 9

Die Turtle soll das rechtsstehende Haus zeichnen. Am Ende soll die Turtle verschwinden. Die Farbe des Hauses lautet sandy brown.



Aufgabe 10

Zeichnen 4 Fünfecke, die jeweils eine andere Farbe haben. Überlegen Sie sich gut, welchen Winkel Sie für das Fünfeck brauchen und welchen Winkel, um 4 davon so zu zeichnen.



Aufgabe 11

Versuchen Sie eine der Flaggen rechts zu zeichnen. Oder eine eigene ausprobieren:)



Lösungen

Lösung 1

```
1 from turtle import *  Py
2 shape("turtle")
3 forward(100)
4 left(90)
5 forward(50)
6 right(90)
7 forward(100)
8 right(90)
9 forward(50)
10 left(90)
11 forward(100)
12 done()
```

Lösung 2

```
1 from turtle import *  Py
2 shape("square")
3 forward(141)
4 right(135)
5 forward(100)
6 right(90)
7 forward(100)
8 done()
```

Lösung 3

```
1 from turtle import *  Py
2 shape("turtle")
3 dot(20)
4 left(45)
5 forward(100)
6 dot(20)
7 right(90)
8 forward(100)
9 dot(20)
10 left(90)
11 forward(100)
12 dot(20)
13 right(90)
14 dot(20)
15 done()
```

Lösung 4

```
1 from turtle import *  Py
2 shape("turtle")
3 left(90)
4 forward(200)
5 left(45)
6 forward(141)
7 left(90)
8 forward(141)
9 left(135)
10 forward(200)
11 right(135)
12 forward(282)
13 left(135)
14 forward(200)
15 left(135)
16 forward(282)
17 left(135)
18 forward(200)
19 done()
```

Lösung 5

```
1 from turtle import *  Py
2 forward(100)
3 right(90)
4 forward(100)
5 left(90)
6 circle(100, 270)
7 left(90)
8 forward(90)
9 penup()
```

```
10 left(90)
11 forward(50)
12 pendown()
13 dot(30)
14 done()
```

Lösung 6

```
1 from turtle import *  Py
2 shape("turtle")
3 pensize(7)
4 pencolor("red")
5 forward(100)
6 right(60)
7 pencolor("yellow")
8 forward(100)
9 right(60)
10 pencolor("pink")
11 forward(100)
12 right(60)
13 pencolor("blue")
14 forward(100)
15 right(60)
16 pencolor("green")
17 forward(100)
18 right(60)
19 pencolor("black")
20 forward(100)
21 right(60)
22 done()
```

Lösung 7

```
1 from turtle import *  Py
2 dot(250)
3 pencolor("red")
4 dot(200)
5 pencolor("blue")
6 dot(150)
7 pencolor("yellow")
8 dot(100)
9 pencolor("pink")
10 dot(50)
11 hideturtle()
12 done()
```

Lösung 8

```
1 from turtle import *  Py
2 pensize(80)
3 left(90)
4 forward(100)
5 pencolor("red")
6 dot(40)
7 penup()
8 back(50)
9 pendown()
10 pencolor("yellow")
11 dot(40)
12 penup()
13 back(50)
14 pendown()
15 pencolor("green")
16 dot(40)
17 hideturtle()
18 done()
```

Lösung 9

```
1 from turtle import *  Py
2 pencolor("sandy brown")
3 fillcolor("sandy brown")
4 begin_fill()
5 forward(141)
6 left(90)
7 forward(141)
8 left(45)
9 forward(100)
10 left(90)
```

```
11 forward(100)
12 left(45)
13 forward(141)
14 end_fill()
15 hideturtle()
16 done()
```

Lösung 10

```
1 from turtle import *  Py
2 pencolor("blue")
3 forward(100)
4 left(72)
5 forward(100)
6 left(72)
7 forward(100)
8 left(72)
9 forward(100)
10 left(72)
11 forward(100)
12 left(72)
13
14 left(90)
15 pencolor("red")
16 forward(100)
17 left(72)
18 forward(100)
19 left(72)
20 forward(100)
21 left(72)
22 forward(100)
23 left(72)
24 forward(100)
25 left(72)
26
27 left(90)
28 pencolor("yellow")
29 forward(100)
30 left(72)
31 forward(100)
32 left(72)
33 forward(100)
34 left(72)
35 forward(100)
36 left(72)
37 forward(100)
38 left(72)
39
40 left(90)
41 pencolor("brown")
42 forward(100)
43 left(72)
44 forward(100)
45 left(72)
46 forward(100)
47 left(72)
48 forward(100)
49 left(72)
50 forward(100)
51 left(72)
52
53 hideturtle()
54 done()
```

Lösung 11

```
1 from turtle import *  Py
2 # Schweizer Flagge
3 speed(5)
4 pencolor("red")
5 fillcolor("red")
6
7 # rotes Quadrat
8 begin_fill()
9 fd(100)
10 lt(90)
11 fd(100)
12 lt(90)
```

```
13 fd(100)
14 lt(90)
15 fd(100)
16 lt(90)
17 end_fill()
18
19 # Verschiebung zum inneren
    Kreuz
20 fd(12.5+25)
21 lt(90)
22 fd(12.5)
23 rt(90)
24 pencolor("white")
25 fillcolor("white")
26
27 # weisses Kreuz
28 begin_fill()
29 fd(25)
30 lt(90)
31 fd(25)
32 rt(90)
33 fd(25)
34 lt(90)
35 fd(25)
36 lt(90)
37 fd(25)
38 rt(90)
39 fd(25)
40 lt(90)
41 fd(25)
42 lt(90)
43 fd(25)
44 rt(90)
45 fd(25)
46 lt(90)
47 fd(25)
48 lt(90)
49 fd(25)
50 rt(90)
51 fd(25)
52 end_fill()
53
54 hideturtle()
55 done()
```